

Análise de Desempenho MPI: Multiplicação Matriz-Vetor

Avaliação de Speedup e Eficiência em Matrizes de Grande Escala

0. 1. Tempos de Execução

A tabela abaixo resume o tempo de execução (em segundos) à medida que aumentamos o tamanho da matriz ($M * N$) e o número de processos.

Processos	16384 x 16384	24576 x 24576	32768 x 32768	40960 x 40960
1	0.52s	1.78s	3.54s	5.22s
2	0.47s	1.65s	3.18s	4.56s
4	0.39s	1.00s	5.68s	3.12s
8	0.35s	1.01s	1.35s	2.61s
16	0.11s	0.35s	0.46s	0.84s
32	0.11s	0.29s	0.47s	0.79s
64	0.11s	0.33s	0.48s	0.78s
128	0.12s	0.34s	0.51s	0.77s

0. 2. Speedup e Eficiência

Diferente do teste inicial com matrizes pequenas, a carga de $40960 * 40960$ (≈ 1.67 bilhão de elementos) proporcionou trabalho suficiente para justificar a paralelização:

- **Speedup Máximo:** O melhor ganho para a matriz 40960×40960 ocorreu com 128 processos, reduzindo o tempo de 5.22s para 0.77s, atingindo um **Speedup** de **6.7x**.
- **Ponto de Saturação:** Para todas as matrizes, observamos um salto gigantesco de desempenho ao migrar de 8 para 16 processos. Contudo, a partir de 32 processos, a curva “achata” (plateau).
- **Eficiência:** A eficiência com 16 processos na matriz maior é de cerca de **38%**. Ao utilizar 128 processos, a eficiência cai para apenas **5.2%**.

0. 3. Conclusão e Gargalos (Communication Bound)

Os dados comprovam que o algoritmo MatVec em MPI é fortemente limitado por banda de memória e comunicação de rede. O ganho computacional linear se esgota rapidamente porque espalhar gigabytes de dados pela rede (via MPI_Scatter e MPI_Bcast) se torna mais lento do que o cálculo de multiplicações pelos núcleos da CPU. Isso demonstra a **Lei de Amdahl** na prática: o tempo gasto na porção estritamente sequencial e na comunicação domina a execução total à medida que P (processos) cresce absurdamente.